

Documentação do Projeto jogoMario

Atividade de Git e GitHub — Organização e versionamento do projeto
Front-End

Nome	Matrícula	Papel
Karllyson Paula	01861379	Testador
Marconi Farias Araújo	01783919	Desenvolvedor
Gustavo Buled Martins	01797483	Documentador
Sérgio Luiz dos Santos Neto	01819969	Desenvolvedor
Pedro Henrique Rodrigues da Silva	01794091	Scrum Master
Janildo Caluête de Farias Júnior	01799458	Testador

Documento elaborado pela área de Documentação do grupo.
2026

1. Sobre o Projeto

O **jogoMario** é uma aplicação Front-End inspirada no universo clássico dos jogos do Mario, desenvolvida em HTML, CSS e JavaScript. O jogo simula um cenário de corrida infinita, no qual o personagem deve pular obstáculos (canos) que se aproximam continuamente pela tela.

Além do desenvolvimento da aplicação, esta atividade tem como propósito colocar em prática o uso de **Git** e **GitHub** como ferramentas de controle de versão e colaboração, organizando o repositório em uma estrutura próxima à utilizada em projetos reais de desenvolvimento de software.

1.1. Objetivo

Exercitar a criação e configuração de um repositório no GitHub, a organização de arquivos em diretórios padronizados, a utilização de branches (**main** e **dev**), a realização de commits significativos e a integração de código por meio de merge, mantendo a aplicação Front-End do jogo funcional e organizada dentro do diretório **frontend/**.

1.2. Como o jogo funciona

- O cenário é composto por um céu com nuvens em movimento, um personagem (Mario) e um cano que se desloca continuamente da direita para a esquerda da tela.
- Ao pressionar qualquer tecla, o personagem executa uma animação de pulo, permitindo desviar do obstáculo.
- O script monitora, em intervalos curtos, a posição do cano e a altura do personagem; caso não haja desvio a tempo, a animação é interrompida e a imagem do personagem é substituída pela tela de Game Over.

2. Integrantes e Papéis

A equipe é composta por seis integrantes, cada um com um papel definido conforme a proposta da atividade. A distribuição foi registrada também no arquivo README.md do repositório.

Nome	Matrícula	Papel
Karlyson Paula	01861379	Testador
Marconi Farias Araújo	01783919	Desenvolvedor
Gustavo Buled Martins	01797483	Documentador
Sérgio Luiz dos Santos Neto	01819969	Desenvolvedor
Pedro Henrique Rodrigues da Silva	01794091	Scrum Master
Janildo Caluête de Farias Júnior	01799458	Testador

Observação: o Scrum Master é responsável pela criação e organização do repositório e pela coordenação das atividades; Documentador(es) cuidam da produção da documentação; Desenvolvedor(es) implementam e evoluem o código; Testador(es) validam o funcionamento e reportam problemas.

3. Estrutura do Repositório

A raiz do repositório segue a estrutura obrigatória definida na atividade, organizada de forma alfabética, contemplando os diretórios de backend, documentação e frontend, além dos arquivos de configuração e licenciamento:

```
jogoMario/  
■■■ backend/  
■   ■■■ .gitkeep  
■■■ docs/  
■   ■■■ branding/  
■   ■   ■■■ .gitkeep  
■   ■■■ mer/  
■   ■   ■■■ .gitkeep  
■   ■■■ mockups/  
■   ■   ■■■ .gitkeep  
■   ■■■ models/  
■   ■   ■■■ uml/  
■   ■       ■■■ .gitkeep  
■   ■■■ requirements/  
■       ■■■ .gitkeep  
■■■ frontend/  
■   ■■■ index.html  
■   ■■■ css/  
■   ■   ■■■ style.css  
■   ■■■ js/  
■   ■   ■■■ script.js  
■   ■■■ images/  
■■■ .gitignore  
■■■ LICENSE  
■■■ README.md
```

3.1. Diretórios de documentação (docs/)

- **branding/** — elementos relacionados à identidade visual do projeto.
- **mer/** — modelos Entidade-Relacionamento.
- **mockups/** — mockups e protótipos das interfaces.
- **models/** — modelos do sistema.
- **models/uml/** — diagramas UML.
- **requirements/** — documentos relacionados aos requisitos.

Como o Git não versiona diretórios vazios, cada diretório sem conteúdo próprio recebe um arquivo **.gitkeep**, garantindo que a estrutura seja preservada no repositório remoto.

4. Tecnologias Utilizadas

Camada	Tecnologia	Finalidade
Estrutura	HTML5	Marcação da tela do jogo (game board).
Estilo	CSS3	Animações (pulo, nuvens, cano) e layout do cenário.
Comportamento	JavaScript (ES6)	Lógica do jogo, detecção de colisão e evento de pulo.
Versionamento	Git	Controle de versão do código-fonte.

Hospedagem/Colaboração no GitHub	Hospedagem do repositório, branches e Pull Requests.
----------------------------------	--

O projeto não depende de frameworks ou bibliotecas externas nem de gerenciador de pacotes: todo o Front-End roda diretamente a partir dos arquivos estáticos contidos em **frontend/**.

5. Instalação e Execução

5.1. Clonar o repositório

```
git clone https://github.com/<usuario-ou-organizacao>/jogoMario.git
cd jogoMario/frontend
```

5.2. Executar o jogo

Como o Front-End é composto apenas por arquivos estáticos (HTML, CSS e JS), não é necessário instalar dependências. Existem duas formas simples de executar o projeto:

- **Opção 1 — Abrir diretamente:** dar duplo clique (ou abrir pelo navegador) no arquivo **frontend/index.html**.
- **Opção 2 — Servidor local (recomendado):** utilizando a extensão **Live Server** do VS Code, ou o comando abaixo dentro de **frontend/**:

```
cd frontend
npx serve .
```

Após abrir a página, pressione qualquer tecla para fazer o personagem pular e desviar do cano que se aproxima.

6. Fluxo de Trabalho — Git e GitHub

O repositório utiliza, no mínimo, duas branches: **main**, que representa a versão principal e integrada do projeto, e **dev**, utilizada para o desenvolvimento contínuo. Todo novo código é primeiro enviado para a **dev** e, após validado, integrado à **main** por meio de merge (ou Pull Request).

Resumo do fluxo adotado:

- 1 Criação do repositório público **jogoMario**, com licença MIT.
- 2 Clonagem do repositório e criação da estrutura de diretórios (backend/, docs/, frontend/).
- 3 Criação da branch **dev** a partir da **main**.
- 4 Desenvolvimento do jogo dentro de **frontend/**, com commits organizados por etapa (ex.: *chore*, *feat*, *docs*, *style*, *fix*).
- 5 Envio da branch **dev** para o GitHub (*git push -u origin dev*).
- 6 Merge da **dev** para a **main**, evidenciando no histórico o desenvolvimento seguido da integração.

6.1. Padrão de commits utilizado

```
chore: cria estrutura inicial do projeto
docs: adiciona README com integrantes do grupo
chore: adiciona .gitignore para Node.js
feat: implementa animação de pulo do personagem
feat: adiciona detecção de colisão com o cano
style: ajusta layout e cores do cenário
fix: corrige posição do personagem ao pular
```

7. Licença

O projeto é disponibilizado publicamente sob a **Licença MIT**, presente no arquivo **LICENSE** na raiz do repositório, permitindo uso, cópia, modificação e distribuição, desde que mantido o aviso de copyright original.

Documento gerado pela equipe responsável pela documentação do projeto jogoMario, como parte da atividade prática de Git e GitHub.